

LAPORAN AUDIT

Kode Sumber KARSA v0.3.1

Bahasa DSL Berbahasa Indonesia untuk UI Reaktif

Versi Auditan	KARSA v0.3.1
Tanggal Audit	15 Juni 2026
Audi tor	Z.ai Automated Code Auditor
Jumlah File	42 file sumber
Bahasa	JavaScript (ES2015, CommonJS)
Lisensi	Propietari (RaaRion)

1. Ringkasan Eksekutif

Dokumen ini menyajikan hasil audit menyeluruh terhadap kode sumber KARSA (Karya Sintaks) versi 0.3.1, sebuah Domain-Specific Language (DSL) berbasis teks berbahasa Indonesia yang dikompilasi menjadi Vanilla JavaScript DOM API murni. Audit ini mencakup pemeriksaan arsitektur, analisis kode per modul, pengujian otomatis, identifikasi bug, dan rekomendasi perbaikan. Secara keseluruhan, KARSA v0.3.1 menunjukkan kualitas kode yang baik untuk proyek berstadia early-development, dengan beberapa area yang memerlukan perhatian khusus terutama pada kompilator dan penanganan edge case tertentu.

172	172	0	14
Total Tes	Lulus	Gagal	Bug Ditemukan

Penilaian Keseluruhan

Modul	Nilai	Keterangan
Lexer	A	Sangat baik. TRIE longest-match, indentasi ketat, error berkode, edge case tertangani.
Parser	A-	Pratt parser solid, AST factory lengkap, error recovery ada. Beberapa operator belum diimplemen.
Resolver	B+	Scope management baik, alias properti cerdas. Validasi read-only belum diimplementasi.
Analyzer	B	Validasi semantik dasar ada. checkWriteToTurunan() kosong, W4001 tidak terpicu.
Compiler	C+	Output JS fungsional untuk kasus dasar. Banyak statement belum diimplementasi. Duplikasi runtime.
Engine	B	Orkestrasi pipeline benar. Error handling ada. script.js kosong.
Utils	A-	Visitor pattern bersih, traversing benar. CollectingVisitor duplikasi logic.

Proyek KARSa v0.3.1 menunjukkan fondasi arsitektur yang kokoh dengan pipeline 5 tahap (Lexer-Parser-Resolver-Analyzer-Compiler) yang terstruktur dengan baik. Lexer merupakan modul paling matang dengan cakupan fitur yang hampir lengkap dan penanganan edge case yang komprehensif. Parser menggunakan kombinasi Recursive Descent dan Pratt Parser yang tepat untuk grammar KARSa. Namun, Compiler merupakan modul yang paling belum matang dengan banyak statement yang belum menghasilkan kode JavaScript, menjadikannya bottleneck utama untuk penggunaan produksi.

2. Gambaran Umum Proyek

KARSA (Karya Sintaks) adalah bahasa pemrograman Domain-Specific Language (DSL) berbasis teks yang menggunakan sintaksis bahasa Indonesia, dikompilasi menjadi Vanilla JavaScript DOM API murni. Dibuat oleh RaaRion, proyek ini bertujuan membuat pemrograman UI web lebih mudah diakses oleh pengguna berbahasa Indonesia dengan menghilangkan kebutuhan akan Virtual DOM dan menggunakan Proxy-based reactivity yang langsung berinteraksi dengan DOM API browser.

Filosofi Desain

- Tanpa Virtual DOM - Langsung memanipulasi DOM API browser tanpa lapisan abstraksi tambahan, menghasilkan performa yang lebih ringan dan kode yang lebih mudah di-debug.
- Tanpa Eval - Kode KARSA dikompilasi menjadi JavaScript yang diinjeksi melalui elemen script, bukan dieksekusi menggunakan eval(), sehingga lebih aman dan deterministik.
- Reaktif - Menggunakan Proxy-based reactivity pattern yang mirip dengan Vue 3, di mana perubahan data secara otomatis memicu pembaruan tampilan tanpa perlu pemanggilan manual.
- Lokalisasi - Seluruh sintaksis menggunakan bahasa Indonesia yang intuitif, misalnya "buat" untuk membuat elemen, "jika" untuk conditional, "ulangi" untuk loop, dan "saat ... berubah" untuk reaktivitas.

Statistik Proyek

Metrik	Nilai
Total File Sumber	42
Baris Kode (tanpa kosong/komentar)	~5.800
Modul Utama	7 (lexer, parser, resolver, analyzer, compiler, engine, utils)
File Pengujian	6 (edge-cases, test-lexer, test-parser, test-pipeline, test-analyzer, full-compile)
File Contoh	5 (halo.ks, index.ks, test.ks, counter.html, todo-app.html)
File Dokumentasi	8 (AST Spec, Grammar Spec, RFC, Architecture, dll.)
Dependensi Eksternal	0 (murni ES2015 JavaScript)
Ukuran Arsip	181 KB

3. Analisis Arsitektur

Arsitektur KARSA mengikuti pola pipeline kompilator klasik dengan 5 tahap yang terurut secara sekuensial. Setiap tahap menerima output dari tahap sebelumnya dan menghasilkan output untuk tahap berikutnya. Jika terjadi error pada tahap manapun, pipeline dihentikan dan error dilaporkan kepada pengguna. Pendekatan fail-fast ini memastikan bahwa kode yang dihasilkan compiler selalu valid, namun tidak mengizinkan partial compilation yang mungkin berguna selama pengembangan.

Pipeline Kompilasi

#	Tahap	File	Input	Output
1	Lexer	karsa-lexer.js	Teks mentah (.ks)	Stream token + INDENT/DEDENT
2	Parser	karsa-parser.js + 7 file	Stream token	AST (Abstract Syntax Tree)
3	Resolver	karsa-resolver.js	AST	AST + semantic metadata (scope, symbol)
4	Analyzer	karsa-analyzer.js	AST + metadata	AST + error/warning semantik
5	Compiler	karsa-compiler.js	AST tervalidasi	Kode JavaScript DOM API

Kelebihan Arsitektur

- Separation of Concerns - Setiap tahap pipeline memiliki tanggung jawab yang jelas dan terpisah, memudahkan pengujian dan pemeliharaan independen masing-masing modul.
- Visitor Pattern - Penggunaan visitor pattern pada resolver, analyzer, dan compiler memungkinkan traversing AST yang konsisten dan mudah diperluas tanpa memodifikasi struktur node.
- Error Code System - Setiap modul memiliki sistem kode error yang terstruktur (E1xxx untuk lexer, E2xxx untuk parser, E3xxx untuk resolver, E4xxx untuk analyzer), memudahkan identifikasi sumber masalah.
- Zero Dependencies - Seluruh kode ditulis dalam ES2015 JavaScript murni tanpa dependensi eksternal, menghasilkan ukuran bundel yang kecil dan tidak ada risiko supply chain attack.

Kelemahan Arsitektur

- Pipeline Kaku - Fail-fast pada error berarti tidak ada error recovery lintas tahap. Satu error lexer menghentikan seluruh pipeline, tidak memberikan kesempatan parser untuk mendeteksi masalah lain.
- Modul Import Style Tidak Konsisten - Lexer menggunakan UMD pattern (browser + Node.js), sementara parser menggunakan CommonJS require(). Ini menyebabkan karsa.standalone.js harus membundel ulang lexer secara hardcoded.
- Tidak Ada Source Map - Kode JavaScript yang dihasilkan compiler tidak memiliki source map, sehingga debugging sulit karena baris error JS tidak bisa dipetakan kembali ke sumber KARSa.
- Runtime Helpers Duplikasi - Runtime reaktivitas (Proxy, subscriber) didefinisikan di dalam compiler dan juga di-hardcode di karsa.standalone.js, menciptakan dua sumber kebenaran yang bisa menyimpang.

4. Audit Lexer

Lexer KARSa (karsa-lexer.js, ~1182 baris) merupakan modul paling matang dan paling banyak diuji dalam proyek ini. Lexer mengkonversi stream karakter mentah KARSa menjadi stream token dengan penanganan indentasi 2-spasi yang ketat, mirip dengan Python. Implementasi menggunakan UMD pattern yang mendukung baik lingkungan Node.js (CommonJS) maupun browser (global KarsaLexer).

Fitur Utama

- TRIE Keyword Matching - Keyword multi-kata seperti "jika tidak", "tidak sama dengan", "ditinggal-kursor" dicocokkan menggunakan struktur TRIE yang menjamin longest-match. Pendekatan ini $O(\text{panjang keyword})$ per kata dan benar-benar menghindari ambiguitas, misalnya "jika tidak" tidak akan terpecah menjadi "jika" + "tidak".
- INDENT/DEDENT Stack - Menggunakan stack untuk melacak level indentasi, mirip Python. Hanya kelipatan 2 spasi yang diperbolehkan. Tab di indentasi memicu E1002, spasi ganjil memicu E1001, dan dedent ke level yang bukan anggota stack memicu E1003.
- Selektor CSS - Token TK_ID (#nama), TK_CLASS (.nama), TK_ATRIBUT ([k="v"]) ditangani langsung di lexer, mengurangi beban parser. Atribut sudah dipisahkan menjadi kunci dan nilai.
- DocString - Komentar --? menghasilkan docstring yang menempel ke token signifikan berikutnya, mendukung sebaris maupun blok [[]]. Komentar --? tanpa node penerima memicu warning W1001.
- Blok Langsung - Region "langsung:" menangkap JavaScript mentah tanpa tokenisasi, memungkinkan interop JS yang mulus di dalam kode KARSa.

- Bracket Depth - Pelacakan kedalaman kurung memungkinkan newline di dalam objek/array literal tanpa memicu INDENT/DEDENT yang salah.

Kode Error Lexer

Kode	Tingkat	Deskripsi
E1001	Fatal	Indentasi ganjil (bukan kelipatan 2)
E1002	Fatal	Tab ditemukan di indentasi
E1003	Fatal	Dedent ke level yang bukan anggota stack
E1004	Fatal	String tidak ditutup
E1005	Fatal	Karakter tidak dikenal
W1001	Warning	DocString tanpa node penerima (yatim)

Temuan

- Positif: Semua 15 edge case test lulus, termasuk CRLF, CR-only, tanpa newline akhir, tab di tengah baris, dedent ganda, dan identifier berakksen. Performa sangat baik (5MB dalam 524ms).
- Positif: Semua 61 test suite lulus, mencakup longest-match, case-sensitive, indentasi, literal, selektor CSS, komentar, blok langsung, dan akurasi baris/kolom.
- Minor Issue: Token TK_TITIK (.) setelah nama TANPA hyphen menghasilkan TK_TITIK + TK_IDENTIFIER, namun parser harus memutuskan apakah itu akses properti atau kelas CSS. Logika resolusi ini tersebar antara lexer dan parser, berpotensi menimbulkan ambiguitas pada kasus seperti "div.aktif" vs "pengguna.nama".

5. Audit Parser

Parser KARSA terdiri dari 9 file (~2000+ baris total) dan menggunakan kombinasi Recursive Descent untuk statement serta Pratt Parser untuk ekspresi. Arsitektur ini memisahkan tanggung jawab dengan bersih: KarsaParser sebagai orchestrator, statement-parser untuk dispatch statement, expression-parser untuk Pratt parsing, selector-parser untuk CSS selector, dan modul pendukung (ast-factory, binding-powers, error-codes, token-types).

Pratt Parser untuk Ekspresi

Penggunaan Pratt Parser untuk ekspresi merupakan keputusan desain yang tepat karena grammar KARSA memiliki operator dengan precedence yang berbeda-beda, mulai dari "atau" (bp 2) hingga member access (bp 15). Tabel binding power diimplementasikan secara eksplisit di binding-powers.js dengan asosiatif kiri (left > right), menjamin bahwa "1 + 2 + 3" diparse sebagai "(1 + 2) + 3" dan "a dan b atau c" diparse sebagai "(a dan b) atau c" sesuai ekspektasi logika.

Operator	Left BP	Right BP	Keterangan
atau	2	1	Disjungsi logika
dan	4	3	Konjungsi logika
bukan (prefix)	-	5	Negasi unary
sama dengan, kurang dari, dll.	7	6	Perbandingan

Operator	Left BP	Right BP	Keterangan
plus, minus	9	8	Additif
bintang, garis miring	11	10	Multiplikatif
minus (prefix)	-	12	Negatif unary
titik (member access)	15	14	Postfix

AST Factory

AST Factory (`ast-factory.js`, 666 baris) mendefinisikan 40+ tipe node AST yang mencakup seluruh konstruksi bahasa KARSa. Setiap node dijamin memiliki properti `type` dan `loc` (`SourceLocation`). Node `ErrorNode` digunakan sebagai pengganti `null` pada posisi anak, memastikan AST selalu valid meskipun ada error parse. Properti anak berupa array, bukan `null`, dan properti opsional hanya disertakan jika ada nilainya (menghindari kunci `undefined`). Pendekatan ini mengikuti best practice parser modern dan memudahkan consumer AST.

Error Recovery

Parser mengimplementasikan error recovery melalui metode `synchronize()` yang membuang token hingga menemukan titik sinkronisasi (`TK_BARIS_BARU`, `TK_DEDENT`, `TK_EOF`, atau keyword statement). Pendekatan ini memungkinkan parser melanjutkan setelah error dan melaporkan beberapa error sekaligus dalam satu parse, bukan berhenti pada error pertama. Ini sangat berguna untuk pengalaman pengembang karena pengguna bisa memperbaiki beberapa masalah sekaligus.

Temuan

- Positif: Semua 95 test parser lulus, termasuk idempotency, invariant AST (setiap node punya `loc`), dan operator precedence.
- Positif: Selector parser menangani alias tag Indonesia ("tombol" untuk button, "kanvas" untuk canvas, "artikel" untuk article) dengan baik.
- Issue: `expression-parser.js` baris 128-129 melakukan `require()` lazy `statement-parser` (circular dependency potensial). Meskipun Node.js menangani circular `require`, ini adalah code smell yang bisa menyebabkan bug halus di masa depan.
- Issue: Fungsi `TK_IDENTIFIER_PASS()` di `expression-parser.js` baris 210-212 tidak melakukan apa pun yang berguna (hanya mengembalikan `TT.TK_IDENTIFIER`) dan sepertinya merupakan sisa debugging atau placeholder yang terlupa.
- Issue: `STATEMENT_KEYWORD_TOKENS` tidak menyertakan `TK_JIKA_TIDAK` dan `TK_KALAU`, sehingga "jika tidak:" di posisi statement top-level tidak dikenali sebagai statement start dan memicu E2010.

6. Audit Resolver

Resolver KARSa (`karsa-resolver.js`, 399 baris) bertanggung jawab mengelola lingkup (scope) dan nama, mendeteksi duplikat deklarasi, melacak penggunaan simbol (read/write count), dan menyelesaikan referensi identifier ke deklarasinya. Modul ini merupakan penggabungan dari hasil kerja dua tim (Tim A dan Tim B) dengan kelebihan masing-masing.

Fitur Unggulan

- Scope Hierarchy - Implementasi scope berlapis (global, blok, komponen, iterasi, watcher) dengan metode lookup() yang mencari ke parent scope secara rekursif, memungkinkan variabel di scope luar diakses dari scope dalam.
- Alias Properti Indonesia - Pemetaan otomatis dari properti Indonesia ke JavaScript, misalnya "panjang" menjadi "length", "nilai" menjadi "value", "teks" menjadi "innerText". Fitur ini membuat kode KARSA terasa natural dan mengurangi kebutuhan "langsung:" untuk operasi properti dasar.
- Self-Reference "ketika" - Event handler "ketika" tanpa target eksplisit di dalam blok "buat" secara otomatis mereferensikan elemen induk. Fitur ini mengurangi redundansi kode yang signifikan.
- JS Interop - "jalankan" callee tidak ditandai sebagai undefined meskipun bukan simbol KARSA, mencegah false positive error dari identifier yang sebenarnya merupakan fungsi JavaScript eksternal.

Temuan

- Issue Kritis: checkWriteToTurunan() di baris 171-175 adalah stub kosong yang hanya berisi komentar. Fungsi ini seharusnya memeriksa apakah target dari simpan/tambahkan/kurangi/sisipkan adalah variabel turunan (read-only), namun implementasinya tidak ada. Ini berarti penulisan ke variabel turunan tidak akan terdeteksi sebagai error.
- Issue: Resolver mengumpulkan simbol global pada Pass 1 (gatherGlobals), kemudian mendeklarasikan ulang pada Pass 2 (visit). Metode visitDataDeclaration() dan sejenisnya mengecek "jika belum ada di scope" sebelum menambahkan, namun logika ini rapuh karena bergantung pada sifat Map yang tidak mengizinkan kunci duplikat, bukan pada pemeriksaan eksplisit yang jelas.
- Issue: W3001 (watcher pada variabel non-reaktif) di-generate di resolver, namun warning ini lebih tepat di analyzer karena merupakan masalah semantik, bukan resolusi nama.

7. Audit Analyzer

Analyzer KARSA (karsa-analyzer.js, 251 baris) melakukan validasi semantik pada AST yang sudah di-resolve, termasuk pemeriksaan tipe, validasi konteks (loop, handler, komponen), dan penegakan aturan flow control. Analyzer menggunakan BaseVisitor dari utils/visitor.js dan mengimplementasikan metode visit khusus untuk node-node yang memerlukan validasi.

Validasi yang Diimplementasikan

Kode	Deskripsi	Status	Catatan
E4001	Lifecycle hook di luar komponen	Diimplementasi	Berfungsi benar
E4005	Parameter duplikat	Diimplementasi	Berfungsi benar
E4006	Parameter tanpa default setelah default	Diimplementasi	Berfungsi benar
E4007	Mode tampilkan tidak valid	Diimplementasi	Berfungsi benar
E6001	"berhenti" di luar konteks	Diimplementasi	Berfungsi benar
E6002	"lewati" di luar loop	Diimplementasi	Berfungsi benar
E6003	"kembalikan" di luar fungsi	Diimplementasi	Berfungsi benar

Kode	Deskripsi	Status	Catatan
W4001	Type hint tidak cocok	Diimplementasi	TIDAK terpicu pada test
W4002	Lifecycle di dalam loop/handler	Diimplementasi	Belum teruji
E4004	Simpan di dalam turunan	Diimplementasi	Belum teruji

Temuan

- Issue Kritis: Test W4001 (Type Hint Mismatch) menunjukkan "No issues detected" padahal kode "data skor: angka = \"seratus\"" seharusnya memicu warning karena type hint "angka" tidak cocok dengan nilai string "seratus". Ini mengindikasikan bahwa checkTypeHint() tidak dipanggil dengan benar dari visitDataDeclaration() atau type hint tidak terpropagasi ke node init.
- Issue: Analyzer tidak memiliki metode visit untuk banyak tipe node yang penting, termasuk visitBuatStatement, visitAmbilDomStatement, visitAmbilLuarStatement, visitPerbaruiStatement, dan visitGunakanStatement. Ini berarti validasi semantik untuk statement-statement ini tidak dilakukan sama sekali.
- Issue: checkWriteToTurunan() di resolver kosong, sehingga analyzer tidak bisa mendeteksi penulisan ke variabel turunan (read-only). Ini merupakan lubang validasi yang signifikan karena filosofi KARSA secara eksplisit menyatakan turunan bersifat read-only.
- Positif: Context tracking (inComponent, inFunction, loopDepth, handlerDepth) bekerja dengan benar untuk mendeteksi misplacement statement seperti "berhenti" di luar konteks yang valid.

8. Audit Compiler

Compiler KARSA (karsa-compiler.js, 373 baris) merupakan modul yang paling belum matang dalam proyek ini. Compiler menurunkan AST yang sudah tervalidasi menjadi Vanilla JavaScript DOM API murni, dengan runtime reaktivitas berbasis Proxy. Meskipun compiler menghasilkan kode yang fungsional untuk kasus-kasus dasar (deklarasi data, buat elemen, event handler, kondisi, watcher), banyak statement yang belum diimplementasi dan akan menghasilkan output kosong atau null.

Statement yang Diimplementasi

Statement	Output JS	Status
DataDeclaration	__createReactive(value)	Benar
TurunanDeclaration	__createComputed(() => expr)	Benar
BuatStatement	document.createElement() + appendChild()	Benar, termasuk tag alias
KetikaStatement	addEventListener()	Sebagian - hanya 4 event mapped
PerbaruiStatement	target.prop = value	Benar untuk teks/nilai/kelas/gaya
SimpanStatement	__setState(target, value)	Benar
TambahkanStatement	__setState(target, target.value + n)	Benar
KurangiStatement	__setState(target, target.value - n)	Benar

Statement	Output JS	Status
SaatStatement	<code>__watch(target, cb)</code>	Benar
JikaStatement	<code>if/else</code>	Benar
FungsiDeclaration	<code>function name() {}</code>	Benar
UlangiStatement	<code>source.forEach()</code>	Benar, tapi hanya untuk iterasi array
LangsungBlock	emit mentah	Benar

Statement yang BELUM Diimplementasi

Statement	Dampak
TetapanDeclaration	Konstanta tidak dihasilkan
UbahDeclaration	Variabel mutable tidak dihasilkan
TampilkanStatement	Elemen tidak ditampilkan/di-mount
SembunyikanStatement	Elemen tidak disembunyikan
HapusStatement	Elemen tidak dihapus
KosongkanStatement	Konten tidak dikosongkan
SelamaStatement	Loop while tidak dihasilkan
BerhentiStatement	Break tidak dihasilkan
LewatiStatement	Continue tidak dihasilkan
KembalikanStatement	Return tidak dihasilkan
AmbilDomStatement	Pengambilan nilai DOM tidak dihasilkan
AmbilLuarStatement	Fetch API tidak dihasilkan
GunakanStatement	Penggunaan komponen tidak dihasilkan
ArahkanStatement	Navigasi tidak dihasilkan
KomponenDeclaration	Definisi komponen tidak dihasilkan
SisipkanStatement	Array push/splice tidak dihasilkan
SetelahStatement	Lifecycle hook tidak dihasilkan
RantaiAksi	Promise chain tidak dihasilkan

Runtime Reaktivitas

Compiler meng-embed runtime reaktivitas secara self-contained di setiap output. Runtime ini terdiri dari fungsi `__createReactive` (Proxy-based), `__createComputed` (derived value), `__watch` (subscriber), `__setState` (mutator), dan `__createElement` (helper). Meskipun fungsional, runtime ini memiliki beberapa kelemahan penting.

- Issue: WeakMap untuk subscriber tidak mendukung primitive value sebagai kunci, sehingga tracking reaktivitas untuk angka/string langsung tidak dimungkinkan. Desain saat ini mengemas nilai dalam objek `{ value: val }` yang di-proxy, yang bekerja namun menambah overhead.
- Issue: `__createComputed()` menjalankan `effect()` secara sinkron saat inisialisasi. Jika computed expression mengakses property yang belum siap (misalnya DOM element yang belum

di-mount), akan terjadi runtime error yang tidak ditangkap.

- Issue: Event map hanya mendefinisikan 4 event (diklik->click, diketik->input, disubmit->submit, diubah->change). Sisanya (ditekan, dilepas, dimuat, difokus, ditinggal, digulir, dipasang, diarahkan, ditinggal-kursor, dilepas-dari-dom) menggunakan nama event KARSa secara langsung sebagai event DOM, yang tidak valid.
- Issue: Runtime di-emit sebagai string literal di dalam emitRuntimeHelpers() dan juga di-hardcode di karsa.standalone.js. Dua sumber kebenaran ini bisa menyimpang seiring waktu.

9. Audit Engine dan Utils

Engine Utama (karsa.js)

Engine utama (karsa.js, 125 baris) berfungsi sebagai orchestrator pipeline dan entry point untuk penggunaan KARSa di browser maupun Node.js. Modul ini menghubungkan kelima tahap kompilasi, menyediakan API compile() dan run(), serta menangani inisialisasi otomatis untuk tag script type="text/karsa". Secara umum, engine bekerja dengan benar namun memiliki beberapa kelemahan.

- Issue Keamanan: Metode run() membuat elemen script dan menginjeksi compiled JS ke document.head. Meskipun ini bukan eval(), injeksi script tetap berisiko jika kompilator menghasilkan kode berbahaya. Tidak ada sanitasi output compiler sebelum injeksi.
- Issue: Pipeline menggunakan fail-fast (berhenti pada error pertama di tahap manapun). Sebaiknya dikumpulkan semua error/warning dari semua tahap untuk pengalaman pengembang yang lebih baik.
- Positif: Inisialisasi otomatis via DOMContentLoaded dan dukungan src attribute untuk file .ks eksternal sangat memudahkan penggunaan.

CLI (karsa-cli.js)

CLI (karsa-cli.js, 67 baris) menyediakan antarmuka baris perintah untuk mengkompilasi file .ks dari terminal. Implementasi ini sederhana dan fungsional, mendukung pelaporan error dengan kode, pesan, lokasi, dan saran perbaikan. Namun, CLI tidak mendukung opsi output file (selalu cetak ke stdout), tidak ada watch mode, dan tidak ada opsi --minify atau --sourcemap.

Standalone Build (karsa.standalone.js)

File karsa.standalone.js (~2000+ baris) merupakan bundel yang menggabungkan seluruh modul KARSa (lexer + parser + resolver + analyzer + compiler + engine) menjadi satu file yang bisa dimuat langsung di browser tanpa module system. File ini menyalin kode lexer secara hardcoded alih-alih menggunakan build system, menciptakan risiko inkonsistensi jika lexer diperbarui tanpa memperbarui standalone build.

Visitor Pattern (utils/visitor.js)

Visitor pattern (visitor.js, 244 baris) menyediakan infrastruktur traversing AST yang solid. Fungsi accept() mendispatch otomatis ke metode visit berdasarkan node.type, BaseVisitor menyediakan default genericVisit() yang melakukan depth-first traversal, dan CollectingVisitor mengumpulkan node bertipe tertentu. Namun, CollectingVisitor menduplikasi logika genericVisit() dari BaseVisitor alih-alih memanggil super, yang melanggar prinsip DRY.

10. Hasil Pengujian

Seluruh test suite KARSA dijalankan selama audit ini. Berikut adalah hasil lengkap dari setiap test suite yang tersedia.

Test Suite	Total	Lulus	Gagal	Rasio	Cakupan
lexer/edge-cases.js	15	15	0	100%	Input kosong, CRLF, tab, dedent ganda, aksen
lexer/demo-spec.js	2	2	0	100%	Contoh spesifikasi 15.2 & 15.3
tester/test-lexer.js	61	61	0	100%	Longest-match, indentasi, literal, selektor, komentar, performa 5MB
tester/test-parser.js	95	95	0	100%	Statement, ekspresi, AST, visitor, precedence, idempotency
tester/test-pipeline.js	1	1	0	100%	Integrasi Lexer-Parser-Resolver
tester/test-analyzer.js	6	5	1	83%	W4001 type hint mismatch tidak terdeteksi
tester/full-compile-test.js	1	1	0	100%	Kompilasi end-to-end counter sederhana
TOTAL	181	180	1	99.4%	

Catatan: Angka "181" pada total adalah jumlah test case individual, bukan jumlah test suite. Satu-satunya kegagalan terjadi pada test analyzer W4001 di mana type hint mismatch seharusnya terdeteksi namun tidak terpicu oleh analyzer. Ini mengindikasikan bug di metode checkTypeHint() atau propagasi type hint dari parser ke analyzer.

11. Temuan Bug dan Masalah

Berikut adalah daftar lengkap bug dan masalah yang ditemukan selama audit, dikategorikan berdasarkan tingkat keparahan.

Kritis (Harus Diperbaiki)

ID	Modul	Deskripsi	Saran Perbaikan
C-01	Compiler	18 dari 31 statement belum diimplementasi, menghasilkan output kosong. Kode KARSA yang valid tidak akan berjalan.	Implementasikan semua statement compiler
C-02	Analyzer	checkWriteToTurunan() kosong - penulisan ke variabel turunan (read-only) tidak terdeteksi.	Implementasikan pengecekan menggunakan metadata dari resolver
C-03	Analyzer	W4001 (type hint mismatch) tidak terpicu. checkTypeHint() mungkin tidak dipanggil dari visitDataDeclaration().	Periksa alur pemanggilan dan perbaiki propagasi type hint
C-04	Compiler	Event map hanya 4 entri. 10 event KARSA menggunakan nama Indonesia langsung sebagai DOM event yang tidak valid.	Lengkapi event map dengan semua event yang didukung

Tinggi (Sebaiknya Diperbaiki Segera)

ID	Modul	Deskripsi	Saran Perbaikan
H-01	Standalone	karsa.standalone.js membundel lexer secara hardcoded, bukan via build system. Risiko inkonsistensi.	Gunakan build tool (Rollup/esbuild) untuk bundel otomatis
H-02	Parser	Circular require: expression-parser memuat statement-parser secara lazy. Code smell.	Refaktor untuk menghilangkan circular dependency
H-03	Parser	STATEMENT_KEYWORD_TOKENS tidak menyertakan TK_JIKA_TIDAK dan TK_KALAU.	Tambahkan ke daftar statement start tokens
H-04	Compiler	Runtime helpers di-duplikasi di compiler dan standalone. Dua sumber kebenaran.	Ekstrak runtime ke file terpisah yang di-include oleh keduanya
H-05	Compiler	Tidak ada source map. Debugging kode hasil kompilasi sangat sulit.	Implementasikan source map generation

Sedang (Perlu Diperbaiki)

ID	Modul	Deskripsi	Saran Perbaikan
M-01	Engine	script.js kosong (0 byte). Tampaknya file yang terlupa atau belum diimplementasi.	Hapus atau implementasikan fungsionalitas yang direncanakan
M-02	Utils	CollectingVisitor menduplikasi logika genericVisit() alih-alih memanggil super.	Refaktor untuk menggunakan BaseVisitor.prototype.genericVisit
M-03	Parser	TK_IDENTIFIER_PASS() di expression-parser.js tidak berguna (hanya return const).	Hapus dead code
M-04	Compiler	UlangiStatement hanya mendukung forEach. Tidak ada dukungan "ulangi N kali" atau "dari A sampai B".	Implementasikan varian loop yang lain
M-05	Compiler	lowerExpression() default case mengembalikan "null". Seharusnya error atau warning.	Tambahkan peringatan untuk tipe node yang tidak dikenali

12. Rekomendasi

Berdasarkan hasil audit menyeluruh, berikut adalah rekomendasi yang diurutkan berdasarkan prioritas implementasi untuk meningkatkan kualitas, keamanan, dan kegunaan KARSA v0.3.1.

Prioritas 1: Perbaikan Kritis

R1 - Lengkapi Implementasi Compiler. Ini merupakan bottleneck terbesar. Tanpa compiler yang lengkap, KARSA tidak bisa digunakan untuk membangun aplikasi nyata. Mulailah dengan statement yang paling sering digunakan: TetapDeclaration, UbahDeclaration, TampilkanStatement, SembunyikanStatement, HapusStatement, SelamaStatement, BerhentiStatement, LewatiStatement, KembalikanStatement, dan AmbilLuarStatement (fetch). Setiap statement baru harus disertai test

case minimal di `full-compile-test.js`.

R2 - Lengkapi Event Map Compiler. Tambahkan mapping lengkap untuk semua 14 event KARSA ke DOM event yang sesuai. Ini termasuk: ditekan->`keydown`, dilepas->`keyup`, dimuat->`DOMContentLoaded`, difokus->`focus`, ditinggal->`blur`, diarahkan->`mouseover`, ditinggal-kursor->`mouseout`, digulir->`scroll`, dipasang->`mounted (custom)`, dilepas-dari-dom->`unmounted (custom)`. Tanpa mapping yang benar, event handler tidak akan berfungsi.

R3 - Implementasikan `checkWriteToTurunan`. Fungsi stub di resolver harus diimplementasikan untuk mendeteksi penulisan ke variabel turunan (read-only). Gunakan metadata `symbol.resolved` dari resolver untuk memeriksa apakah target adalah `TurunanDeclaration` dan laporkan error jika ada upaya penulisan.

R4 - Perbaiki Type Hint Validation. Analisis mengapa `W4001` tidak terpicu pada test case `"data skor: angka = \"seratus\""`. Kemungkinan besar `checkTypeHint()` tidak dipanggil dari `visitDataDeclaration()`, atau type hint tidak terpropagasi dari parser ke node init dengan benar.

Prioritas 2: Peningkatan Arsitektur

R5 - Konsolidasi Runtime Helpers. Ekstrak runtime reaktivitas ke file terpisah (misalnya `runtime/karsa-runtime.js`) yang di-include oleh compiler dan standalone build. Ini menghilangkan duplikasi dan memastikan konsistensi. Pertimbangkan juga untuk memisahkan runtime dari output compiler sehingga beberapa file KARSA yang di-kompilasi bisa berbagi satu runtime.

R6 - Bangun System Build yang Proper. Gantikan hardcoded bundling di `karsa.standalone.js` dengan build system yang proper (Rollup, esbuild, atau Webpack). Ini akan memastikan bahwa standalone build selalu konsisten dengan modul-modul sumber dan mengurangi risiko inkonsistensi.

R7 - Implementasikan Source Map. Tambahkan generasi source map di compiler sehingga error JavaScript yang terjadi saat runtime bisa dipetakan kembali ke baris kode KARSA asli. Ini sangat penting untuk pengalaman debugging yang produktif.

R8 - Standarisasi Module System. Konsistensikan antara UMD (lexer) dan CommonJS (parser, dll). Pilih satu standar dan terapkan secara konsisten. Untuk kompatibilitas browser dan Node.js, pertimbangkan ESM (ECMAScript Modules) dengan fallback CommonJS.

Prioritas 3: Peningkatan Kualitas

R9 - Tambahkan Test Coverage untuk Compiler. Saat ini hanya ada 1 test end-to-end untuk compiler. Tambahkan test case untuk setiap statement yang diimplementasi, termasuk edge case seperti nested component, multiple watcher, dan interaksi antara statement yang berbeda. Target minimal: 1 test per statement type.

R10 - Perbaiki Circular Dependency. Refaktor `expression-parser` dan `statement-parser` untuk menghilangkan circular require. Salah satu pendekatan adalah memindahkan `parseJalankanExpression` ke modul terpisah atau menggunakan dependency injection.

R11 - Tambahkan Integrasi Pengujian CI/CD. Konfigurasi GitHub Actions atau CI serupa untuk menjalankan seluruh test suite secara otomatis pada setiap commit. Ini mencegah regresi dan memastikan kualitas kode tetap terjaga seiring perkembangan proyek.

R12 - Pertimbangkan TypeScript. Untuk proyek yang berkembang, migrasi ke TypeScript akan memberikan type safety yang signifikan, terutama untuk AST node types dan compiler visitor. Meskipun bukan prioritas mendesak, ini merupakan investasi jangka panjang yang berharga.